

CHAPTER 15

IMPLEMENTATION



15.6 Integration

- The approach up to now:
 - Implementation followed by integration
- This is a poor approach
- Better:
 - Combine implementation and integration methodically

Product with 13 Modules

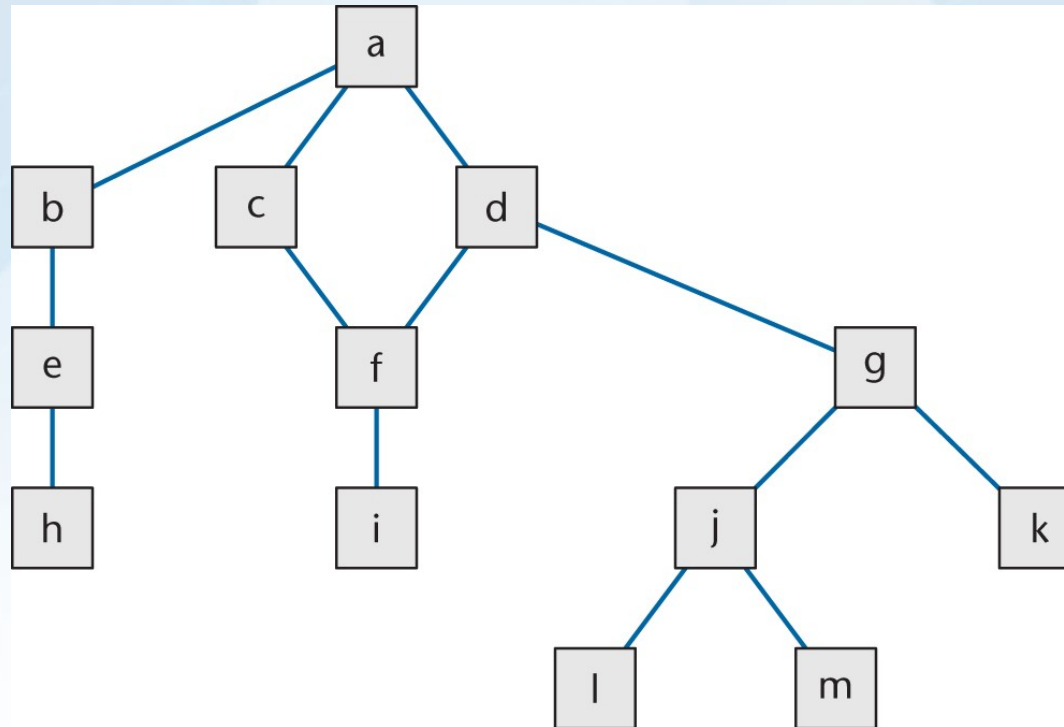


Figure 15.6

Implementation, Then Integration

- Code and test each code artifact separately
- Link all 13 artifacts together, test the product as a whole
- Stub: Empty artifact
- Driver: A code artifact that calls the artifact under test one or more times

Drivers and Stubs

- To test artifact a , artifacts b, c, d must be stubs
 - An empty artifact, or
 - Prints a message, or
 - Returns value corresponding to preplanned test cases
- To test artifact h on its own requires a driver, which calls it
 - Once, or
 - Several times, or
 - Many times, each time checking the value returned
- Testing artifact d requires a driver and two stubs

Implementation, Then Integration (contd)

- Problem 1
 - Stubs and drivers must be written, then thrown away after unit testing is complete
- Problem 2
 - Lack of fault isolation
 - A fault could lie in *any* of the 13 artifacts or 13 interfaces
 - In a large product with, say, 103 artifacts and 108 interfaces, there are 211 places where a fault might lie
- Solution to both problems
 - Combine unit and integration testing

15.6.1 Top-down Integration

- If code artifact `mAbove` sends a message to artifact `mBelow`, then `mAbove` is implemented and integrated before `mBelow`
- One possible top-down ordering is
 - a, b, c, d, e, f, g, h, i, j, k, l, m

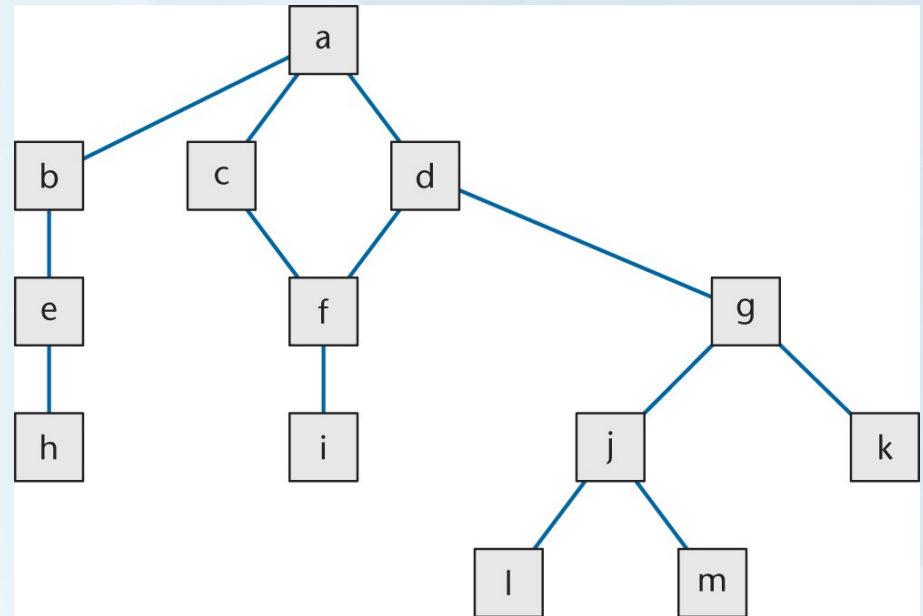


Figure 15.6 (again)

15.6.1 Top-down Integration (contd)

- Advantage 1: Fault isolation
 - A previously successful test case fails when m_{New} is added to what has been tested so far
 - The fault must lie in m_{New} or the interface(s) between m_{New} and the rest of the product
- Advantage 2: Stubs are not wasted
 - Each stub is expanded into the corresponding complete artifact at the appropriate step

15.6.2 Bottom-up Integration

- If code artifact m_{Above} calls code artifact m_{Below} , then m_{Below} is implemented and integrated before m_{Above}
- One possible bottom-up ordering is
l, m, h, i, j, k, e,
f, g, b, c, d, a

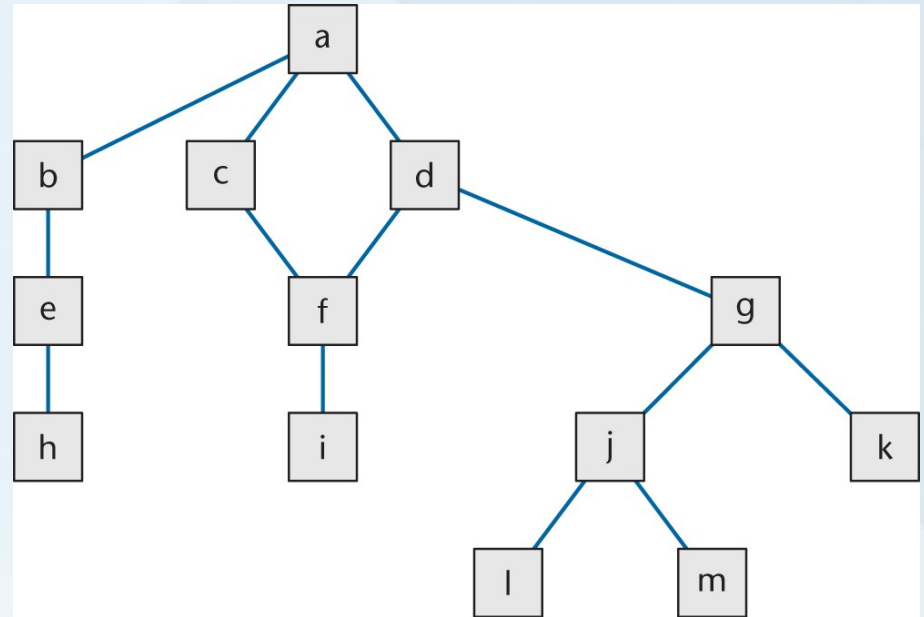


Figure 15.6 (again)

15.6.2 Bottom-up Integration (contd)

- Advantage 1
 - Operational artifacts are thoroughly tested
- Advantage 2
 - Operational artifacts are tested with drivers, not by fault shielding, defensively programmed artifacts
- Advantage 3
 - Fault isolation

15.6.2 Bottom-up Integration (contd)

- Difficulty 1
 - Major design faults are detected late
- Solution
 - Combine top-down and bottom-up strategies making use of their strengths and minimizing their weaknesses

15.6.3 Sandwich Integration

- Logic artifacts are integrated top-down
- Operational artifacts are integrated bottom-up
- Finally, the interfaces between the two groups are tested

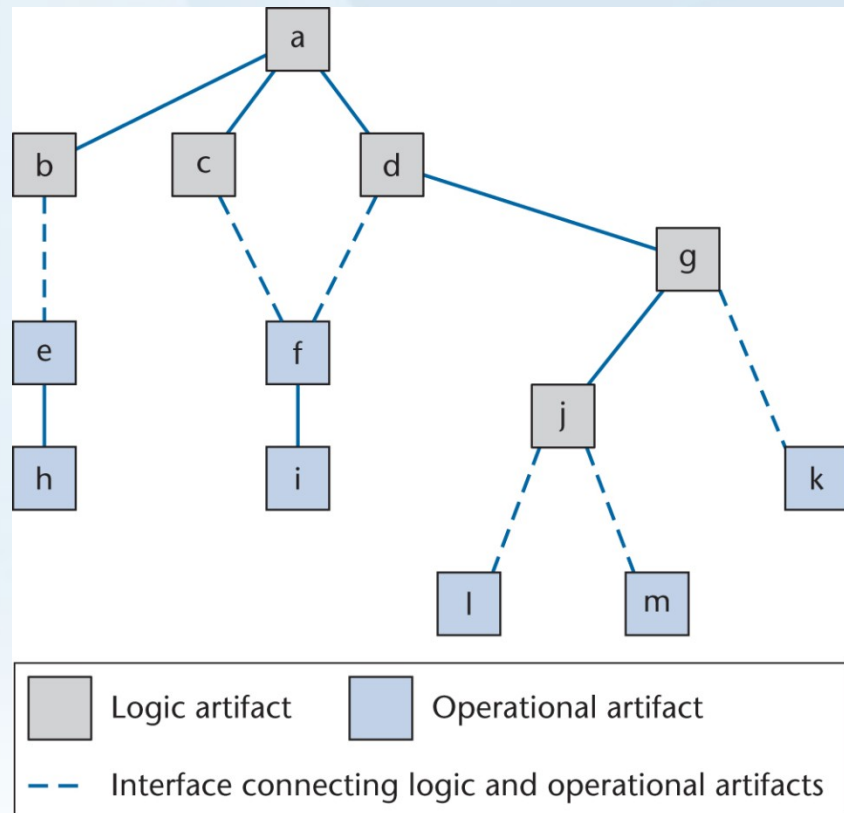


Figure 15.7

15.6.1 Top-down Integration (contd)

- Logic artifacts include the decision-making flow of control
- Operational artifacts perform the actual operations of the product
- The logic artifacts are implemented and integrated before the operational artifacts

15.6.3 Sandwich Integration (contd)

- Advantage 1
 - Major design faults are caught early
- Advantage 2
 - Operational artifacts are thoroughly tested
 - They may be reused with confidence
- Advantage 3
 - There is fault isolation at all times

15.11 Black-Box Unit-testing Techniques

- Neither exhaustive testing to specifications nor exhaustive testing to code is feasible
- The art of testing:
 - Select a small, manageable set of test cases to
 - Maximize the chances of detecting a fault, while
 - Minimizing the chances of wasting a test case
- Every test case must detect a previously undetected fault

15.11.1 Equivalence Testing and Boundary Value Analysis

- Example
 - The specifications for a DBMS state that the product must handle any number of records between 1 and 16,383 ($2^{14} - 1$)
 - If the system can handle 34 records and 14,870 records, then it probably will work fine for 8,252 records
- If the system works for any one test case in the range (1..16,383), then it will probably work for any other test case in the range
 - Range (1..16,383) constitutes an *equivalence class*

Equivalence Testing

- Any one member of an equivalence class is as good a test case as any other member of the equivalence class
- Range (1..16,383) defines three different equivalence classes:
 - Equivalence Class 1: Less than 1 record
 - Equivalence Class 2: From 1 through 16,383 records
 - Equivalence Class 3: More than 16,383 records

Boundary Value Analysis

- Select test cases on or just to one side of the boundary of equivalence classes
 - _ This greatly increases the probability of detecting a fault

(see next slide for example)

Equivalence Testing and Boundary Value Analysis

- Test case 1: 0 records Member of equivalence class 1 and adjacent to boundary value
- Test case 2: 1 record Boundary value
- Test case 3: 2 records Adjacent to boundary value
- Test case 4: 723 records Member of equivalence class 2
- Test case 5: 16,382 records Adjacent to boundary value
- Test case 6: 16,383 records Boundary value
- Test case 7: 16,384 records Adjacent to boundary value
- Test case 8: 19000 records Member of equivalence class 3

